Tutorial 8:

Object Detection using Faster RCNN (Custom Model)

Objectives:

- Understand Faster RCNN
- Understand ROI & pooling
- Intersection of Union (IOU)
- Mean Average Precision (mAP)

Here's a step-by-step breakdown:

Step 1 —Import Libraries

We will import the important libraries to get started.

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import cv2
import matplotlib.pyplot as plt
import random
```

Step 2 Construct the Backbone

We'll use a simple Convolutional Neural Network (CNN) as our feature extractor.

```
# Step 1: Define the Backbone Network (Simple CNN)
def build_backbone(input_shape=(224, 224, 3)):
    inputs = keras.Input(shape=input_shape)
    x = keras.layers.Conv2D(64, (3, 3), activation='relu', padding='same')(inputs)
    x = keras.layers.MaxPooling2D((2, 2))(x)
    x = keras.layers.Conv2D(128, (3, 3), activation='relu', padding='same')(x)
    x = keras.layers.MaxPooling2D((2, 2))(x)
    x = keras.layers.Conv2D(256, (3, 3), activation='relu', padding='same')(x)
    x = keras.layers.MaxPooling2D((2, 2))(x)
    outputs = keras.layers.Conv2D(512, (3, 3), activation='relu', padding='same')(x)
    return keras.Model(inputs, outputs)

backbone = build_backbone()
```

Step 3: Constructing the Region Proposal Network (RPN):

The RPN predicts regions of interest (ROIs) using anchors. Here, we use two sets of output layers:

- Objectness score: Predicts whether an anchor contains an object or not.
- Bounding box regression: Adjusts the anchor boxes to fit the object more precisely.

```python
# Step 2: Build the Region Proposal Network (RPN)
def build_rpn(feature_map):
    rpn_conv = keras.layers.Conv2D(512, (3, 3), padding="same", activation="relu")(feature_map)
    rpn_class = keras.layers.Conv2D(9 * 2, (1, 1), activation="sigmoid")(rpn_conv)  # 9 anchors, 2 classes (object/non-object)
    rpn_bbox = keras.layers.Conv2D(9 * 4, (1, 1))(rpn_conv)  # 9 anchors, 4 bounding box values
    return rpn_class, rpn_bbox
```

Step 4: Implement the ROI pooling layer

The ROI Pooling Layer extracts fixed-size feature maps from the ROIs.

```python
# Step 3: Implement the ROI Pooling Layer
class ROIPoolingLayer(tf.keras.layers.Layer):
    def __init__(self, pool_size, **kwargs):
        super(ROIPoolingLayer, self).__init__(**kwargs)
        self.pool_size = pool_size

    def call(self, inputs):
        feature_map, rois = inputs
        rois = tf.reshape(rois, (-1, 4))
        batch_indices = tf.zeros((tf.shape(rois)[0],), dtype=tf.int32)
        pooled_rois = tf.image.crop_and_resize(
            feature_map, boxes=rois, box_indices=batch_indices, crop_size=self.pool_size
        )
        return pooled_rois

    def compute_output_shape(self, input_shape):
        feature_map_shape, rois_shape = input_shape
        num_rois = rois_shape[0]
        return (num_rois, self.pool_size[0], self.pool_size[1], feature_map_shape[-1])

roi_pooling_layer = ROIPoolingLayer((7, 7))
```

Step 5: Create Classification and Regression Heads

After pooling features, we create dense layers to classify objects and predict bounding box adjustments.

```
# Step 4: Create Classification and Regression Heads
def build_heads(pooled_rois, num_classes):
    flatten = keras.layers.Flatten()(pooled_rois)
    dense = keras.layers.Dense(1024, activation="relu")(flatten)
    classifier = keras.layers.Dense(num_classes, activation="softmax")(dense)
    bbox_regressor = keras.layers.Dense(num_classes * 4)(dense)  # 4 values per bounding box
    return classifier, bbox_regressor
```

Step 6: Assembling the Complete Faster R-CNN Model:

We bring all the components together to build the final Faster R-CNN model.

```
# Step 5: Assemble the Complete Faster R-CNN Model
def build_faster_rcnn(num_classes, input_shape=(224, 224, 3)):
    input_image = keras.Input(shape=input_shape)
    rois = keras.Input(shape=(None, 4))  # ROIs input

    # Backbone (feature extraction)
    backbone = build_backbone(input_shape)
    feature_map = backbone(input_image)

    # RPN
    rpn_class, rpn_bbox = build_rpn(feature_map)

    # ROI Pooling
    pooled_rois = roi_pooling_layer([feature_map, rois])

    # Classification & Regression heads
    classifier, bbox_regressor = build_heads(pooled_rois, num_classes)

    # Build the complete model
    model = keras.Model(inputs=[input_image, rois], outputs=[rpn_class, rpn_bbox, classifier, bbox_regressor])
    return model
```

Step 7: Printing Summary of the model:

The summary of the model can be displayed via

```
# Instantiate the model
num_classes = 80  # Example number of classes for COCO dataset
faster_rcnn = build_faster_rcnn(num_classes)
faster_rcnn.summary()
```

Step 8: Calculating IOU:

To Calculate the IOU

```python
# Step 6: Calculate Intersection over Union (IoU)
def calculate_iou(box1, box2):
    ymin1, xmin1, ymax1, xmax1 = box1
    ymin2, xmin2, ymax2, xmax2 = box2
    inter_xmin = max(xmin1, xmin2)
    inter_ymin = max(ymin1, ymin2)
    inter_xmax = min(xmax1, xmax2)
    inter_ymax = min(ymax1, ymax2)
    inter_area = max(0, inter_xmax - inter_xmin) * max(0, inter_ymax - inter_ymin)
    area1 = (xmax1 - xmin1) * (ymax1 - ymin1)
    area2 = (xmax2 - xmin2) * (ymax2 - ymin2)
    union_area = area1 + area2 - inter_area
    iou = inter_area / union_area if union_area != 0 else 0
    return iou
```

Step 9: Calculating Precision & Recall

```python
# Step 7: Calculate Precision and Recall
def calculate_precision_recall(pred_boxes, pred_scores, pred_labels, gt_boxes, gt_labels, iou_threshold=0.5):
    tp, fp = 0, 0
    matched_gt = set()
    for i, pred_box in enumerate(pred_boxes):
        if pred_scores[i] < iou_threshold:
            continue
        best_iou = 0
        best_gt_index = -1
        for j, gt_box in enumerate(gt_boxes):
            if pred_labels[i] == gt_labels[j] and j not in matched_gt:
                iou = calculate_iou(pred_box, gt_box)
                if iou > best_iou:
                    best_iou = iou
                    best_gt_index = j
        if best_iou >= iou_threshold:
            tp += 1
            matched_gt.add(best_gt_index)
        else:
            fp += 1
    fn = len(gt_boxes) - len(matched_gt)
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0
    return precision, recall
```

Step 10: Calculating Average Precision & mAP

```python
# Step 8: Calculate Average Precision (AP)
def calculate_ap(precisions, recalls):
    precisions = [0.0] + precisions + [0.0]
    recalls = [0.0] + recalls + [1.0]
    for i in range(len(precisions) - 1, 0, -1):
        precisions[i - 1] = max(precisions[i - 1], precisions[i])
    indices = [i for i in range(1, len(recalls)) if recalls[i] != recalls[i - 1]]
    ap = sum((recalls[i] - recalls[i - 1]) * precisions[i] for i in indices)
    return ap

# Step 9: Calculate Mean Average Precision (mAP) Across All Classes
def calculate_map(predictions, ground_truths, num_classes, iou_threshold=0.5):
    average_precisions = []
    for class_id in range(1, num_classes):  # Assuming class_id 0 is background
        all_precisions = []
        all_recalls = []
        for pred, gt in zip(predictions, ground_truths):
            pred_boxes = [box for i, box in enumerate(pred['boxes']) if pred['labels'][i] == class_id]
            pred_scores = [score for i, score in enumerate(pred['scores']) if pred['labels'][i] == class_id]
            gt_boxes = [box for i, box in enumerate(gt['boxes']) if gt['labels'][i] == class_id]
            precision, recall = calculate_precision_recall(
                pred_boxes, pred_scores, [class_id] * len(pred_boxes), gt_boxes, [class_id] * len(gt_boxes), iou_threshold
            )
            all_precisions.append(precision)
            all_recalls.append(recall)
        ap = calculate_ap(all_precisions, all_recalls)
        average_precisions.append(ap)
    mAP = np.mean(average_precisions)
    return mAP
```

Step 11: Sample Values for Obtaining mAP

```python
# Placeholder predictions and ground truths for testing
predictions = [
    {'boxes': [[0.1, 0.1, 0.4, 0.4], [0.5, 0.5, 0.8, 0.8]], 'scores': [0.9, 0.7], 'labels': [1, 2]}
]
ground_truths = [
    {'boxes': [[0.12, 0.12, 0.42, 0.42], [0.5, 0.5, 0.8, 0.8]], 'labels': [1, 2]}
]

# Number of classes in the dataset (e.g., 80 for COCO)
num_classes = 80

# Calculate mAP
mean_ap = calculate_map(predictions, ground_truths, num_classes)
print(f"Mean Average Precision (mAP): {mean_ap:.2f}")
```

Tasks:

- Change the number of layers
- Take a data if you have and just randomly test it what it shows
- Do a labelling of about 150 images through labelling tools and develop your own model train and test on the data and show the results.

How to find Average Precision:

The **area under the Precision–Recall (PR) curve** is what we call **Average Precision (AP)**. It is calculated by **integrating precision over recall.**

$$AP = \int_0^1 P(R)\, dR$$

Since we usually have **discrete detection points**, the area is computed **numerically** rather than analytically.

Numerical Area Calculation (Interpolation Method)

The area is approximated using **rectangular integration**:

$$AP = \sum (R_n - R_{n-1}) \times P_n$$

where

- R_n = recall at step n
- P_n = precision at step n

Recall	Precision
0.1	1.00
0.2	0.90
0.4	0.85
0.6	0.80
0.8	0.75
1.0	0.70

Example

Using the table above:

Interval	Calculation
0 → 0.1	$(0.1 - 0) \times 1.0 = 0.10$
0.1 → 0.2	$(0.2 - 0.1) \times 0.9 = 0.09$
0.2 → 0.4	$(0.4 - 0.2) \times 0.85 = 0.17$
0.4 → 0.6	$(0.6 - 0.4) \times 0.80 = 0.16$
0.6 → 0.8	$(0.8 - 0.6) \times 0.75 = 0.15$
0.8 → 1.0	$(1.0 - 0.8) \times 0.70 = 0.14$

Now sum:

$$AP = 0.10 + 0.09 + 0.17 + 0.16 + 0.15 + 0.14$$

$$AP = 0.81$$